

Implementation of Peer-to-Peer Energy Trading via Solana Smart Contracts in a Permissioned Environment

Chanthawat Kiriyaadee¹, and Suwannee Adsavakulchai^{1*}

¹Department of Computer Engineering and Artificial Intelligence, School of Engineering, University of the Thai Chamber of Commerce (UTCC), Bangkok, Thailand

Contact Email: 2410717302003@live4.utcc.ac.th, Suwannee_ads@utcc.ac.th

Thailand's three state utilities — EGAT, MEA, and PEA — have pursued a National Energy Trading Platform (NETP) since 2017 for peer-to-peer (P2P) trading among rooftop-solar prosumers, but it remains in development, and the regulated buy-back affords prosumers no direct means of exchanging surplus within the local distribution grid. This paper evaluates a settlement layer for that role: six modular Anchor programs on a permissioned Solana runtime in which every high-frequency write is routed to a per-entity program-derived account (PDA), so transactions of distinct entities hold disjoint write sets and are scheduled in parallel by the Sealevel engine, ordered by Proof-of-History [1]. We measure compute, throughput and latency together, and find that they are not bound by the same thing. Compute is $O(1)$ and never binds: the steady telemetry write holds 13.3–13.6 k compute units (CU) across a $2,500\times$ sweep from 80 to 200,000 meters and against 510,000 resident PDAs, while confirmation latency stays flat at $p50 \approx 1.5\text{--}1.9$ s ($p95 \leq 3.1$ s). Throughput is instead **associated** with lock footprint: ordering every measured path by the shared write-locked state it touches spans three orders of magnitude where the CU column does not — yet controlled experiments do not confirm the association as causal. Restoring the prefix lock footprint (the parent ZoneMarket re-declared writable, one account flag) leaves settlement throughput statistically unchanged from one fee payer to eight, and a previously reported ≈ 0.6 TPS proves to be a serial-await harness artifact. At ≈ 120 k CU a settle exhausts the block compute budget before its locks bind, so the write-set lever is not visible below a multi-node scheduler.

1. Introduction

Since 2017 Thailand's state utilities have pursued a National Energy Trading Platform (NETP) for P2P trading among rooftop-solar prosumers, implemented on a Tendermint-based consortium blockchain [2]. This article presents a Solana-based architecture as an alternative execution layer, motivated by NETP's own evaluation, in which Tendermint outperformed both Ethereum and Hyperledger Fabric under throughput testing: that result suggests further gains from a runtime whose execution is parallel rather than sequential. Such evaluations report throughput and compute cost as though computation were the axis along which a settlement layer scales. The contribution here is to separate the axes: partition all hot-path state per entity, then measure compute, throughput and latency independently, so that what binds each can be named rather than assumed.

2. The Per-Entity Partition

The settlement layer is six Anchor programs [3] (anchor-lang and anchor-spl 1.0.0) — **energy-token**, **governance**, **oracle**, **registry**, **trading**, and **treasury** — in Rust, compiled to SBF bytecode. Programs invoke one another through cross-program invocation (CPI), authority delegated to PDAs that sign without any private key. A PDA is the SHA-256 of its seed tuple, bump byte, program id and a domain-separation constant — accepted only if **not** a valid Ed25519 curve point, so no private key can exist and the owning program may sign as that address. A transaction declares in advance every account it touches and whether each is writable; the runtime locks per account, so two transactions with disjoint writable sets run on different cores and two writing one account serialise. Table 1 gives the hot-path partition under that rule.

Table 1: Hot-path state partition. Every high-frequency write targets its own program-derived account, so any two are Sealevel-parallelisable.

Hot path	Account	Seed tuple
meter telemetry	MeterState	[b"meter", meter_id] — the 32-byte id ceiling is the runtime's own seed limit
order placement	Order	[b"order", authority, order_id]
settlement replay guard	OrderNullifier	[b>nullifier", user, order_id]
registration counters	RegistryShard $\times 16$	[b"registry_shard", shard_id, shard = key[0] mod 16]

Two consequences follow. First, global accounts — config, totals — are read-only on hot paths and **deliberately stale**; periodic admin instructions fold per-entity state back into them. Second, derivation is itself a budget item: find_program_address searches the bump byte downward from 255, a hash plus a curve-decompression check per candidate. The programs store the winning bump and thereafter validate with a single create_program_address — **1,651 CU against 12,136** on the aggregation path, a $7.3\times$ saving on pure addressing.

3. Method

All experiments ran on one node (Apple M2, Agave 3.1.10) of a private solana-test-validator network, so they characterise SVM semantics — account locking and compute metering — not consensus or leader rotation [4]. Throughput is **client-observed confirmed goodput**: confirmed transactions per wall-clock second from burst start to last confirmation, covering the pipeline from client signing to confirmation visibility. Every non-confirmed transaction is attributed to one class — send-rejected (refused by the RPC node, no fee), guard-rejected (executed, failed a program check, fee paid and recorded on-chain), or expired — separating delivery loss from validation working as intended. Transactions are pre-signed against a cached blockhash and submitted raw without preflight or retry, then confirmed by bulk signature-status polling on a 1.5 s sweep under a 3,000-transaction in-flight window; latency is therefore poll-quantised (± 1.5 s), an upper bound on the runtime's own cost rather than a measurement of it. CU is machine-independent, read post-hoc from transaction metadata. The telemetry sweep submits one reading per meter from N distinct meters over two epochs at least 61 s apart — the rate-limit and monotonic-timestamp guards force the gap — separating one-off PDA creation from the recurring steady write. It covers $N = 80$ to 200,000 (1.35 M transactions) over two runs, the second unreset between scales (Table 3). The order-entry burst submits $N = 10^4$ orders over 16 round-robin zone shards on the same transport and fee payer, making the two directly comparable.

4. Results and Discussion

Compute is $O(1)$ and never binds. Table 2 gives the instruction profile against the 200,000-CU default per-instruction budget; every control, telemetry and settlement-recording path costs at most $\approx 8\%$ of it. Table 3 resolves the telemetry write

by fleet size: 13,337–13,634 CU from 80 to 200,000 meters, the entire 2,500× range spanned by 297 CU ($\pm 1.1\%$), two independent runs agreeing to within 1.4% at every shared scale. The residual tracks meter-identifier byte length, not N ; the first write for a meter costs 16,050 CU (15,957–16,254 live) because it initialises and rent-funds the PDA. Under a concurrency sweep from 5 to 40 in-flight transactions cost is likewise flat at 22–24 k CU, so execution is load-independent as well as fleet-independent. Settlement is the one expensive instruction at 121,813 CU (60.9%), dominated by native Ed25519 verification and instruction-sysvar introspection authorising the trade, not by settlement arithmetic.

Table 2: **Compute cost per instruction**, against the 200,000-CU default per-instruction budget. Machine-independent, read from transaction metadata.

Instruction	CU	% of 200 k
do_nothing (dispatch + signature-verify floor)	769	0.4%
treasury.record_settlement	3,301	1.7%
treasury.record_settlement_sharded	5,374	2.7%
trading.submit_limit_order_sharded	9,884	4.9%
oracle.submit_meter_reading (steady)	13,376	6.7%
oracle.submit_meter_reading (first, inits PDA)	16,050	8.0%
trading.settle_offchain_match	121,813	60.9%

Table 3: **Per-meter compute against fleet size**. Steady-state CU for oracle.submit_meter_reading, measured live. Runs A and B are independent sweeps; B ran its scales back-to-back without a ledger reset, so its 200,000-meter case writes against 310,000 resident PDAs and ends holding 510,000.

Fleet size N (meters)	80	1,000	10,000	50,000	100,000	150,000	200,000
Steady write, run A (CU)	13,560	13,634	13,337	13,337	13,634	—	—
Steady write, run B (CU)	—	—	13,525	13,525	13,560	13,560	13,560

Compute is also $O(1)$ in accumulated state: run B’s last 200,000 submissions cost the same against 510,000 resident accounts as the first scale’s did against 10,000. The account store explains it: an update appends to an append-only file rather than rewriting in place, and lookup goes through an in-memory pubkey $\rightarrow$ (slot, offset) index, so either write costs one probe and one append. What scales with account count — index memory, snapshot size, startup time — is a multi-node concern, invisible to per-transaction cost.

Throughput orders by lock footprint, but the mechanism does not confirm. Table 4 sorts every measured path by the shared writable state it touches, and that ordering — not the CU column — tracks the rate across three orders of magnitude. The causal test fails. Anchor’s mut imposes a writability **requirement**, not a prohibition, so the shipped binary accepts the pre-fix lock footprint when the client declares ZoneMarket writable — one AccountMeta, verified on the wire. Over five repeats of 1,200 settles, arm order alternating, the writable arm returns 493 against 561 confirmed settles·s⁻¹ across eight fee payers and 447 against 446 across one. Even the shared fee-payer lock does not move throughput: conflicting transactions still land within one slot as successive entries, so a write lock costs wall-clock drain, not slot occupancy — and at ≈ 120 k CU per settle the block compute budget saturates before locks bind. We therefore withdraw the 2.7–3× previously attributed to deleting this mut; both original figures are accounted for: a session’s first burst runs ≈ 5 – $6\times$ slower than every later one (validator warm-up), and the ≈ 0.6 TPS once reported for single-payer settlement is a serial send-and-await harness measuring its own round-trip (0.69 TPS, p50 463 ms reproduced). Order entry shares the caveat: the **cheaper** instruction (9,884 CU against 13,337) returned 180 TPS against 462 under a vestigial mut since dropped; the path has not been re-measured, and the lock is no longer established as its cause. Telemetry, holding only the gateway fee payer, is flat at 426–490 TPS from 5,000 to 200,000 meters: not proportional, since one payer signs everything, but not degrading, since per-meter PDAs never contend.

Table 4: **Throughput by lock footprint**. Paths ordered by the shared write-locked state they touch — an ordering, not a demonstrated cause (see text). Harnesses differ per row; rows are not mutually comparable. *pre-fix* = measured before the vestigial mut on zone_market was dropped.

Path	CU	TPS	Shared writable state
RPC read	—	$\approx 1,454$	none — no consensus round-trip
meter ingest (5 k–200 k meters)	13.3–13.6 k	426–490	gateway fee payer
order entry (10 k orders, pre-fix)	9,884	180	fee payer + zone_market
OLTP proxy (concurrency 5 $\rightarrow$ 40)	22–24 k	7.9 $\rightarrow$ 74.3	fee payer + market accounts
settlement, concurrent, 1–8 fee payers	≈ 120 k	446–561	fee payer + energy_mint + collectors

Latency is set by block time, not by the partition. Confirmation holds p50 ≈ 1.5 – 1.9 s and p95 ≤ 3.1 s across the whole 2,500× fleet range — flat where a contended design would degrade — and order entry sits at p50 2,173 ms, p95 3,565 ms (measured pre-fix). Both are send $\rightarrow$ first-seen-confirmed upper bounds dominated by the ≈ 400 – 600 ms single-node block time, not measurements of runtime cost — visible in the read path, which needs no consensus round-trip and returns in under a millisecond, three orders of magnitude faster than any write. **Delivery loss is zero** across the 1.02 M first-attempt sends of the unreset sweep; the 0.46–0.48% residue is a deterministic anomaly-gate probe firing on the same meter indices every epoch, and since the rate-limit and monotonic-timestamp guards make resubmission idempotent, one retry round would put true loss near 10^{-5} .

5. Conclusion

With hot-path state partitioned per entity, neither computation nor per-entity contention bounds a P2P energy settlement layer: the steady write is $O(1)$ in fleet size and in resident account count, and latency is flat over a 2,500× sweep. Lock footprint orders the measured paths more closely than instruction cost, but the ordering did not survive a controlled test as a mechanism: settlement throughput is unchanged from one fee payer to eight — at ≈ 120 k CU the settle exhausts the block compute budget before its locks bind — and the two strongest prior data points dissolve into validator warm-up and a serial-await harness. Shrinking declared write sets remains sound design; below a multi-node scheduler, a before/after on lock footprint chiefly measures its own harness. A single validator bounds execution and locking only; reproducing these results on a multi-validator consortium mapped to EGAT, MEA, and PEA would place both consensus and the scheduler under test, and suits UEC–ASEAN collaboration.

Acknowledgment

The authors thank the University of the Thai Chamber of Commerce (UTCC) for institutional support, and the organizers, the UEC ASEAN Research Center (UARC) and Multimedia University (MMU), for hosting this workshop.

References

- [1] A. Yakovenko, “Solana: A New Architecture for a High Performance Blockchain v0.8.13.” 2018.
- [2] Metropolitan Electricity Authority (MEA) and Electricity Generating Authority of Thailand (EGAT) and Provincial Electricity Authority (PEA), “Development of the National Energy Trading Platform (NETP): Research and Development Project Report,” technical report, Dec. 2019.
- [3] Coral / Anchor contributors, “Anchor: Solana’s Sealevel runtime framework.” [Online]. Available: <https://www.anchor-lang.com/>
- [4] Solana Foundation, “Tower BFT: Solana’s High Performance Implementation of PBFT.” Jul. 2019.